

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Dataset Integration

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO3: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.

Assessment Tool Weightage: 30%

Total Marks: 20

Code of Conduct

I **Anto Chriswin G 192324093** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Dataset Integration Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Dataset Understanding & Preparation	30%	Correctly understands both datasets, identifies relevant columns/keys, and loads them properly.	Minor misunderstanding or preparation issue.	Loads datasets but needs some guidance in identifying relevant fields.	Unable to identify or prepare the datasets correctly.
Dataset Integration	20%	Correctly integrates datasets using appropriate merge(), join(), or concat() and selects the correct integration key.	Integration is mostly correct with minor errors.	Performs integration with significant guidance or minor key/operation errors.	Unable to integrate the datasets correctly.
Data Processing & Analysis	15%	Correctly performs all required calculations, filtering, grouping, or transformations	Performs most required analysis correctly.	Performs basic analysis but misses some required operations.	Analysis is incorrect or incomplete.

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
		on the integrated data.			
Python/Pandas Implementation	20%	Code is correct, efficient, well-structured, and uses appropriate Pandas functions.	Code works with minor inefficiencies/errors.	Code works partially and requires corrections.	Code contains major errors or does not execute.
Output & Interpretation	15%	Clearly presents the required output and correctly interprets the results.	Output is correct with limited interpretation.	Output is partially correct or poorly presented.	Output is missing/incorrect and results are not interpreted.

Total Marks Awarded:

Signature of the Course Faculty

Questions:

Set 2 – Dataset Integration Tasks

1. Customer–Order Integration

You are given:

- customer.csv — Customer_ID, Customer_Name, City
- orders.csv — Order_ID, Customer_ID, Order_Date, Order_Amount

Task: Merge the datasets using Customer_ID. Display customer name, city, order date, and order amount. Find the customer with the highest total order amount.

```
1 import pandas as pd
2
3 customer = pd.DataFrame({
4     "Customer_ID": ["C101", "C102", "C103", "C104", "C105"],
5     "Customer_Name": ["Arun", "Priya", "Rahul", "Sneha", "Karthik"],
6     "City": ["Chennai", "Bangalore", "Hyderabad", "Mumbai", "Chennai"]
7 })
8
9
10 orders = pd.DataFrame({
11     "Order_ID": ["O001", "O002", "O003", "O004", "O005",
12                 "O006", "O007", "O008", "O009", "O010"],
13     "Customer_ID": ["C101", "C102", "C101", "C103", "C104",
14                    "C102", "C105", "C103", "C101", "C105"],
15     "Order_Date": ["2026-08-01", "2026-08-02", "2026-08-03",
16                    "2026-08-04", "2026-08-05", "2026-08-06",
17                    "2026-08-07", "2026-08-08", "2026-08-09",
18                    "2026-08-10"],
19     "Order_Amount": [2500, 1800, 3200, 4500, 2100,
20                      2700, 5000, 3500, 1500, 4200]
21 })
22
23 merged = pd.merge(customer, orders, on="Customer_ID")
24
25 print("Merged Data:")
26 print(merged[["Customer_Name", "City", "Order_Date", "Order_Amount"]])
27
28 total = merged.groupby("Customer_Name")["Order_Amount"].sum()
29
30 highest = total.idxmax()
31
32 print("\nCustomer with highest total order amount:")
33 print(highest)
34 print("Total Amount:", total.max())
```

Output :

```
Merged Data:
  Customer_Name  City  Order_Date  Order_Amount
0         Arun  Chennai  2026-08-01           2500
1         Arun  Chennai  2026-08-03           3200
2         Arun  Chennai  2026-08-09           1500
3        Priya  Bangalore  2026-08-02           1800
4        Priya  Bangalore  2026-08-06           2700
5        Rahul  Hyderabad  2026-08-04           4500
6        Rahul  Hyderabad  2026-08-08           3500
7        Sneha   Mumbai  2026-08-05           2100
8       Karthik  Chennai  2026-08-07           5000
9       Karthik  Chennai  2026-08-10           4200
```

```
Customer with highest total order amount:
Karthik
Total Amount: 9200
```

2. Employee–Attendance Integration

You are given:

- employee.csv — Employee_ID, Employee_Name, Department
- attendance.csv — Employee_ID, Date, Status

Task: Integrate the two datasets using Employee_ID. Calculate the number of days each employee was present and absent.

```
1 import pandas as pd
2
3 employee = pd.DataFrame({
4     "Employee_ID": ["E101", "E102", "E103", "E104", "E105"],
5     "Employee_Name": ["Arun", "Priya", "Rahul", "Sneha", "Karthik"],
6     "Department": ["IT", "HR", "Finance", "IT", "Sales"]
7 })
8
9 attendance = pd.DataFrame({
10     "Employee_ID": [
11         "E101", "E101", "E101",
12         "E102", "E102", "E102",
13         "E103", "E103", "E103",
14         "E104", "E104", "E104",
15         "E105", "E105", "E105"
16     ],
17     "Date": [
18         "2026-08-01", "2026-08-02", "2026-08-03",
19         "2026-08-01", "2026-08-02", "2026-08-03",
20         "2026-08-01", "2026-08-02", "2026-08-03",
21         "2026-08-01", "2026-08-02", "2026-08-03",
22         "2026-08-01", "2026-08-02", "2026-08-03"
23     ],
24     "Status": [
25         "Present", "Present", "Absent",
26         "Present", "Absent", "Present",
27         "Absent", "Present", "Present",
28         "Present", "Present", "Absent",
29         "Present", "Absent", "Absent"
30     ]
31 })
32
33 merged = pd.merge(employee, attendance, on="Employee_ID")
34
35 result = merged.groupby(
36     ["Employee_ID", "Employee_Name", "Department", "Status"]
37 ).size().unstack(fill_value=0)
38
39 result = result.rename(columns={
40     "Present": "Present_Days",
41     "Absent": "Absent_Days"
42 })
43
44 print("Attendance Summary:")
45 print(result.reset_index())
```

Output :

```
Attendance Summary:
Status Employee_ID Employee_Name Department Absent_Days Present_Days
0          E101          Arun          IT           1           2
1          E102          Priya          HR           1           2
2          E103          Rahul    Finance           1           2
3          E104          Sneha          IT           1           2
4          E105        Karthik        Sales           2           1
```

3. Course–Student Integration

You are given:

- students.csv — Student_ID, Student_Name, Department
- enrollment.csv — Student_ID, Course_ID, Course_Name, Credits

Task: Merge the datasets using Student_ID. Display student name, department, course name, and credits. Find the total number of credits registered by each student.

```
1 import pandas as pd
2
3 students = pd.DataFrame({
4     "Student_ID": ["S101", "S102", "S103", "S104", "S105"],
5     "Student_Name": ["Arun", "Priya", "Rahul", "Sneha", "Karthik"],
6     "Department": ["CSE", "IT", "ECE", "CSE", "IT"]
7 })
8
9 enrollment = pd.DataFrame({
10     "Student_ID": [
11         "S101", "S101",
12         "S102", "S102",
13         "S103", "S103",
14         "S104", "S104",
15         "S105", "S105"
16     ],
17     "Course_ID": [
18         "C101", "C102",
19         "C103", "C104",
20         "C105", "C106",
21         "C101", "C107",
22         "C103", "C108"
23     ],
24     "Course_Name": [
25         "Python", "Database",
26         "Java", "Data Science",
27         "Computer Networks", "Operating Systems",
28         "Python", "Machine Learning",
29         "Java", "Cloud Computing"
30     ],
31     "Credits": [4, 3, 4, 3, 4, 3, 4, 4, 4, 3]
32 })
33
34 merged = pd.merge(students, enrollment, on="Student_ID")
35
36 print("Merged Data:")
37 print(merged[[
38     "Student_Name",
39     "Department",
40     "Course_Name",
41     "Credits"
42 ]])
43
44 total_credits = merged.groupby(
45     ["Student_ID", "Student_Name"]
46 ).["Credits"].sum()
47
48 print("\nTotal Credits Registered by Each Student:")
49 print(total_credits)
```

Output:

Merged Data:

	Student_Name	Department	Course_Name	Credits
0	Arun	CSE	Python	4
1	Arun	CSE	Database	3
2	Priya	IT	Java	4
3	Priya	IT	Data Science	3
4	Rahul	ECE	Computer Networks	4
5	Rahul	ECE	Operating Systems	3
6	Sneha	CSE	Python	4
7	Sneha	CSE	Machine Learning	4
8	Karthik	IT	Java	4
9	Karthik	IT	Cloud Computing	3

Total Credits Registered by Each Student:

Student_ID	Student_Name	
S101	Arun	7
S102	Priya	7
S103	Rahul	7
S104	Sneha	8
S105	Karthik	7

Name: Credits, dtype: int64

4. Sales–Region Integration

You are given:

- sales.csv — Sale_ID, Product_ID, Region_ID, Quantity, Amount
- regions.csv — Region_ID, Region_Name, Manager

Task: Integrate the datasets using Region_ID. Display the region name, manager, product ID, quantity, and amount. Calculate the total sales amount for each region.

```
1 import pandas as pd
2 sales = pd.DataFrame({
3     "Sale_ID": ["S001", "S002", "S003", "S004", "S005",
4     "S006", "S007", "S008", "S009", "S010"],
5     "Product_ID": ["P101", "P102", "P103", "P104", "P105",
6     "P101", "P102", "P103", "P104", "P105"],
7     "Region_ID": ["R01", "R02", "R01", "R03", "R02",
8     "R03", "R01", "R02", "R03", "R01"],
9     "Quantity": [10, 8, 5, 12, 10, 7, 6, 9, 8, 4],
10    "Amount": [5000, 4000, 3000, 6000, 5500,
11    3500, 2800, 4500, 4200, 2200]
12 })
13 regions = pd.DataFrame({
14     "Region_ID": ["R01", "R02", "R03"],
15     "Region_Name": ["North", "South", "West"],
16     "Manager": ["Arun", "Priya", "Rahul"]
17 })
18 merged = pd.merge(sales, regions, on="Region_ID")
19 print("Merged Data:")
20 print(merged[["
21     "Region_Name",
22     "Manager",
23     "Product_ID",
24     "Quantity",
25     "Amount"
26 ]])
27 total_sales = merged.groupby("Region_Name")["Amount"].sum()
28
29 print("\nTotal Sales Amount for Each Region:")
30 print(total_sales)
```

Output:

```
Merged Data:
  Region_Name Manager Product_ID Quantity Amount
0      North   Arun      P101        10   5000
1      South  Priya      P102         8   4000
2      North   Arun      P103         5   3000
3       West  Rahul      P104        12   6000
4      South  Priya      P105        10   5500
5       West  Rahul      P101         7   3500
6      North   Arun      P102         6   2800
7      South  Priya      P103         9   4500
8       West  Rahul      P104         8   4200
9      North   Arun      P105         4   2200

Total Sales Amount for Each Region:
Region_Name
North      13000
South      14000
West       13700
Name: Amount, dtype: int64
```

5. Weather–City Integration

You are given:

- cities.csv — City_ID, City_Name, State
- weather.csv — City_ID, Date, Temperature, Humidity, Rainfall

Task: Merge the datasets using City_ID. Display city name, state, date, temperature, humidity, and rainfall. Identify the city with the highest average temperature.

```
1 import pandas as pd
2
3 cities = pd.DataFrame({
4     "City_ID": ["C101", "C102", "C103", "C104", "C105"],
5     "City_Name": ["Chennai", "Bangalore", "Hyderabad", "Mumbai", "Delhi"],
6     "State": ["Tamil Nadu", "Karnataka", "Telangana", "Maharashtra", "Delhi"]
7 })
8
9 weather = pd.DataFrame({
10     "City_ID": [
11         "C101", "C101",
12         "C102", "C102",
13         "C103", "C103",
14         "C104", "C104",
15         "C105", "C105"
16     ],
17     "Date": [
18         "2026-08-01", "2026-08-02",
19         "2026-08-01", "2026-08-02",
20         "2026-08-01", "2026-08-02",
21         "2026-08-01", "2026-08-02",
22         "2026-08-01", "2026-08-02"
23     ],
24     "Temperature": [32, 34, 28, 30, 35, 36, 31, 33, 37, 38],
25     "Humidity": [75, 70, 65, 60, 55, 50, 70, 68, 45, 42],
26     "Rainfall": [10, 5, 8, 4, 2, 1, 12, 7, 0, 0]
27 })
28
29 merged = pd.merge(cities, weather, on="City_ID")
```

```

1  print("Merged Data:")
2  print(merged[[
3      "City_Name",
4      "State",
5      "Date",
6      "Temperature",
7      "Humidity",
8      "Rainfall"
9  ]])
10 average_temp = merged.groupby("City_Name")["Temperature"].mean()
11
12 print("\nAverage Temperature of Each City:")
13 print(average_temp)
14 highest_city = average_temp.idxmax()
15 highest_temperature = average_temp.max()
16
17 print("\nCity with Highest Average Temperature:")
18 print(highest_city)
19 print("Average Temperature:", highest_temperature)

```

Output:

```

Merged Data:
  City_Name      State      Date  Temperature  Humidity  Rainfall
0  Chennai  Tamil Nadu  2026-08-01         32         75         10
1  Chennai  Tamil Nadu  2026-08-02         34         70          5
2  Bangalore  Karnataka  2026-08-01         28         65          8
3  Bangalore  Karnataka  2026-08-02         30         60          4
4  Hyderabad  Telangana  2026-08-01         35         55          2
5  Hyderabad  Telangana  2026-08-02         36         50          1
6   Mumbai  Maharashtra  2026-08-01         31         70         12
7   Mumbai  Maharashtra  2026-08-02         33         68          7
8    Delhi      Delhi  2026-08-01         37         45          0
9    Delhi      Delhi  2026-08-02         38         42          0

```

Average Temperature of Each City:

City_Name

Bangalore 29.0

Chennai 33.0

Delhi 37.5

Hyderabad 35.5

Mumbai 32.0

Name: Temperature, dtype: float64

City with Highest Average Temperature:

Delhi

Average Temperature: 37.5